

Project Status & Guidelines: HR & Access Control System

Project State Summary (As of May 2026)

The project currently encompasses a solid infrastructural backend based on an overarching architecture design:

- 1. Database Layer (PostgreSQL - BCNF Normalized)**
 - Schema implemented correctly fulfilling Boyce-Codd Normal Form requirements (`database/init.sql`).
 - Extended to encompass Security & Authentication constraints (Role-Based Access Control configuration utilizing three special tables (`ROLES` , `USER_CREDENTIALS` , `EMPLOYEE_ROLES`)).
 - Fully containerized inside `docker-compose.yml` .
- 2. Backend API Layer (FastAPI, Python)**
 - Base folder structure and ORM entities set up mirroring relational designs (`models.py` , `schemas.py`).
 - Essential logic mapped into RESTful Endpoints inside `main.py` mimicking the system matrix.
 - JWT-based OAuth2 security protocol successfully integrated into an `auth.py` router mechanism preventing vertical privilege escalation on all vital endpoints (Admin/HR exclusive access, Cross-Verification for IDs on Access Logs).
 - Docker configuration prepared and successfully built structure ready to run via `docker-compose` .
- 3. Frontend Layer (TypeScript / Next.js 16 / React 19)**
 - Fully scaffolded **Next.js 16** application using the **App Router** (`src/app/`) with **React 19** and **TypeScript**.
 - **Authentication flow implemented:** `AuthContext` provider manages JWT token storage (`localStorage`), user fetch via `/auth/me` , and exposes `login/logout` helpers through a custom `useAuth` hook.
 - **Route protection implemented:** `ProtectedRoute` wrapper component redirects unauthenticated users to `/login` and renders an "Unauthorized" message when the user's roles don't match `allowedRoles` .
 - **Pages built:**
 - `/login` — Email/password form that POSTs `FormData` to `/auth/login` , stores the JWT, and redirects to `/dashboard` .
 - `/dashboard` — Sidebar navigation (Dashboard, Employees, Access Logs, Leave Requests) with role-conditional links, user info, and logout button. Uses `lucide-react` icons.
 - `/` — Server-side redirect to `/dashboard` .
 - **Data-fetching layer ready:** `Providers.tsx` wraps the app in `QueryClientProvider` from **TanStack React Query v5**.
 - **Type contracts defined:** `types/index.ts` (Schedule, Department, Employee, LeaveRequest, AccessLog, Payslip) and `types/auth.ts` (AuthUser, AuthContextState).
 - **Styling:** Tailwind CSS v4 via PostCSS; Google Fonts (Geist, Geist Mono).
 - **Dockerized:** `Dockerfile` + `docker-compose.yml` service with `NEXT_PUBLIC_API_URL` env var.
 - **Pending:** Sub-pages for Employees list/detail, Access Logs monitor, and Leave Request management have navigation links but their route pages (`/dashboard/employees` , `/dashboard/access-logs` , `/dashboard/leave-requests`) are **not yet created**.

Technical & Lingual Guidelines

1. Unified Language Protocol (English-Only)

- **Codebase Standardization:** All source code, including variables, classes, database table names, SQL commands, function hooks, error strings, and comments **must be exclusively written in English**.
- **Documentation:** All documentation files (`.md` files), schema plans, user manuals, deployment templates, and commit messages **must be generated in English**.
- **Internal API communication:** API returned messages such as custom HTTP 404/400 detail errors or console outputs must be entirely in English (e.g. *"Employee not found"*, *"Cannot sequentially Check-In twice"*).
- *Failure to uphold this requirement across future modules invalidates the repository's consistency policies.*

2. Architecture Constraints

- **Docker-First Environment:** Development instances must always deploy through `docker-compose`. Hardcoded `localhost` variables should be shifted to docker's networking layers where needed for proper routing mechanisms.
- **Relational Operations:** Backend routing structures must rigorously use `SQLAlchemy` ORM queries over raw strings unless explicitly required, ensuring cascade rules map properly to the relational constraints.
- **Role-Based Adherence:** No data mutation endpoint should be exposed unprotected. The frontend must implement `AuthContext` to intercept and reroute unauthenticated operations mimicking the rules already created for the Backend.
- **Type-Safety Contracts:** TypeScript interface components for API interaction must perfectly match the endpoint `schemas.py` signatures avoiding runtime casting errors.

3. Current Directory Structure

To help navigate the workspace, here is the current file layout:

```
/
├── docker-compose.yml          # Main docker-compose configuration (db, backend,
                                frontend)
├── database/
│   └── init.sql                # Initial SQL script (Schema + Auth tables)
├── backend/
│   ├── Dockerfile              # Python API Docker container config
│   ├── requirements.txt         # Python dependencies
│   └── app/
│       ├── auth.py             # JWT, Password hashing, RBAC logic
│       ├── database.py          # SQLAlchemy DB connection setup
│       ├── main.py              # FastAPI application and endpoints
│       ├── models.py            # ORM Models mapping for database
│       └── schemas.py           # Pydantic schemas for request/response
├── validation
├── frontend/
│   ├── Dockerfile              # Next.js Docker container config
│   ├── package.json            # Dependencies (Next 16, React 19, TanStack
                                Query, Axios, etc.)
│   ├── next.config.ts          # Next.js configuration
│   ├── tsconfig.json           # TypeScript configuration
│   ├── postcss.config.mjs      # PostCSS / Tailwind v4 configuration
│   └── types/
│       └── index.ts             # Entity interfaces (Employee, Department,
                                Schedule, etc.)
```

```
| | └─ auth.ts # Auth interfaces (AuthUser, AuthContextState)
| | └─ context/
| | └─ AuthContext.tsx # JWT auth provider with login/logout/useAuth
hook
| | └─ components/
| | | └─ ProtectedRoute.tsx # RBAC route guard component
| | └─ src/app/
| | | └─ globals.css # Tailwind imports & CSS variables
| | | └─ layout.tsx # Root layout (Providers + AuthProvider wrapper)
| | | └─ Providers.tsx # TanStack React Query client provider
| | | └─ page.tsx # Root redirect → /dashboard
| | | └─ login/
| | | | └─ page.tsx # Login form (email/password → JWT)
| | | └─ dashboard/
| | | | └─ page.tsx # Main dashboard (sidebar + welcome card)
└─ md/
    └─ Auth & Security modifications.md
    └─ Project Status and Guidelines.md
    └─ Project_Blueprint_DB_Backend_Frontend.md
    └─ Project_db_normalization_of_tables.md
    └─ Table Structure ER Diagram.md
```